

Effortlessly Manage Image Downloads with Redis Queue: A Step-by-Step Guide

 systemdesignschool.io/fundamentals/redis-queue-tutorial



Redis Queue Tutorial

Here is a simple example using Redis Queue (RQ) to download an image from the internet, which can be a time-consuming task. Here's how the components interact with each other:

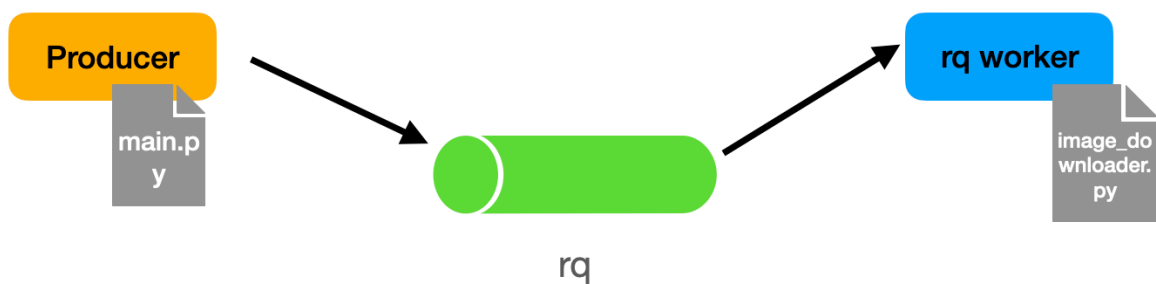


Image download task using Redis Queue

For quick practice, we provide a Docker environment:

```
docker run -dit --name redis-queue-practice systemdesignschool/redis-queue-tutorial tail -f /dev/null
```

```

~ %>docker run -dit --name redis-queue-practice systemdesignsch
ool/redis-queue-tutorial tail -f /dev/null
Unable to find image 'systemdesignschool/redis-queue-tutorial:latest' locally
latest: Pulling from systemdesignschool/redis-queue-tutorial
3610fc6cdcdb: Already exists
440ff9e33d74: Already exists
0dc0d0d23e1f: Already exists
e9246f522e03: Already exists
7d0edd549123: Already exists
ae793ad4e179: Already exists
6660d8dc96d3: Already exists
a83e100ee346: Already exists
90324b79189b: Already exists
613041c9feaf: Already exists
Digest: sha256:9d8589baea2e610d49a98445247444cb35809a571ddf4471b163e8b6f710c867
Status: Downloaded newer image for systemdesignschool/redis-queue-tutorial:lates
t
a3c19d155b4e3177cea718184080f98ebfe41ea3635da854218e7f888a79cdc1

```



NAME ↓		TAG	IMAGE ID	CREATED	SIZE
systemdesignschool/redis-queue-tutorial	IN USE	latest	5946ab90cc23	16 days ago	363.21 MB



The `systemdesignschool/redis-queue-tutorial` Docker image includes `Redis`, `Python`, and the required scripts for practice (`image_downloader.py`, `main.py`).

If you choose to set up the environment yourself, you need to install these components. The Python code will be provided below.

Now open the Docker container interactive environment and run Redis:

```

~ %>docker exec -it redis-queue-practice bash root@ec6c6307531d:/app# redis-
server 13:C 11 Sep 2025 08:45:36.670 * o000o000o000o Redis is starting
o000o000o000o 13:C 11 Sep 2025 08:45:36.670 * Redis version=8.0.2, bits=64,
commit=00000000, modified=0, pid=13, just started 13:C 11 Sep 2025 08:45:36.670
# Warning: no config file specified, using the default config. In order to
specify a config file use redis-server /path/to/redis.conf 13:M 11 Sep 2025
08:45:36.672 * monotonic clock: POSIX clock_gettime __ __-`__ '-__ __-`__
`_ '-__ Redis Open Source .-` .-` .-` \ _ _ '-__ 8.0.2 (00000000/0) 64
bit ( ' , .-` | ` ) Running in standalone mode |`-__-__-` __-__-`-__-`-__-`
'| Port: 6379 | `-.__ `-. / _-' | PID: 13 `-.__ `-. / _-' _-' |`-__-__-
`-.__-` _-' _-' | | `-.__-` _-' _-' | https://redis.io `-.__ `-.__-` _-' _-'
 _-' |`-__-__-` _-' _-' | | `-.__-__-` _-' _-' | `-.__ `-.__-` _-' _-'
 _-' `-.__ `-.__-` _-' _-' `-.__-` _-' _-' 13:M 11 Sep 2025 08:45:36.675 *
Server initialized 13:M 11 Sep 2025 08:45:36.675 * Ready to accept connections
tcp

```

Open another terminal and enter the Docker container interactive environment to check that Redis starts clean. `redis-cli KEYS '*'` confirms we indeed have no data in Redis.

```

~ %>docker exec -it redis-queue-practice bash root@ec6c6307531d:/app# redis-cli
KEYS '*' (empty array)

```

View files in the directory:

```
root@ec6c6307531d:/app# ls image_downloader.py main.py requirements.txt
```

You can see that currently there are only `image_downloader.py`, `main.py`, and `requirements.txt`.

First, `image_downloader.py` is used to download images and save them to files. The code is as follows:

```
import requests
def download_image(url):
    response = requests.get(url)
    with open(url.split("/")[-1], "wb") as f:
        f.write(response.content)
    print(f"Image downloaded and saved")
```

Next, `main.py` creates the Redis queue, imports the function, and enqueues the task. The code is as follows:

```
from redis import Redis
from rq import Queue
from image_downloader import download_image
q = Queue(connection=Redis())
q.enqueue(download_image, "http://www.lenna.org/lena_std.tif", )
```

`requirements.txt` defines the libraries that the Python code depends on:

```
rq==2.6.0 requests==2.32.5 redis==6.4.0
```

Next, run the script:

```
root@ec6c6307531d:/app# python main.py
```

You won't see anything in this terminal, but we can check the data in Redis:

```
root@ec6c6307531d:/app# redis-cli KEYS '*'
1) "rq:job:27148712-d3c5-4762-bc8d-8af71001320f"
2) "rq:queue:default"
3) "rq:queues"
```

We now have tasks in our message queue! Time to get some workers to get the job done.

Start a worker and you should see something like this:

```
root@ec6c6307531d:/app# rq worker
09:52:39 Worker 4f16aada3ca34f2c9b8812486f3854b1: started with PID 32, version 2.6.0
09:52:39 Worker 4f16aada3ca34f2c9b8812486f3854b1: subscribing to channel
rq:pubsub:4f16aada3ca34f2c9b8812486f3854b1 09:52:39 *** Listening on default...
09:52:39 Worker 4f16aada3ca34f2c9b8812486f3854b1: cleaning registries for queue: default
09:52:39 default:
image_downloader.download_image('http://www.lenna.org/lena_std.tif') (27148712-d3c5-4762-bc8d-8af71001320f) Image downloaded and saved
09:52:41 Successfully completed image_downloader.download_image('http://www.lenna.org/lena_std.tif')
job in 0:00:02.107133s on worker 4f16aada3ca34f2c9b8812486f3854b1
09:52:41 default: Job OK (27148712-d3c5-4762-bc8d-8af71001320f)
09:52:41 Result is kept for 500 seconds
```

The worker completed the job successfully and we can see our image in the local directory.

Exit `rq worker` and check the downloaded image `lenna_std.tif`:

```
root@ec6c6307531d:/app# ls __pycache__ image_downloader.py lena_std.tif main.py requirements.txt
```

Amazing! You just had a taste of how message queues work!

This is a simple example with a rather small image. But imagine the possibilities:

- You could enqueue heavier tasks such as hash encoding a video, converting a video to another format, etc., that could take a long time.
- You could have many workers when the load gets high.
- And many other things you can do with a message queue.

Feel free to explore the documentation for more features and examples at <https://python-rq.org>.